

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

TRABAJO PRÁCTICO INTEGRADOR

Food Store: Sistema de Gestión de Pedidos de Comida

Materia: Programación III

Alumno: Damián Ignacio Nogueira

Coordinador: Ariel Enferrel

Profesores: Cinthia Rigoni y Alberto Cortez

Comisión: M25 C3-02

Fecha de entrega: 29 de junio de 2026

Año lectivo: 2026

ÍNDICE

1. Introducción	3
2. Marco Teórico	3 - 4
3. Decisiones Técnicas y Arquitectura	4 - 6
4. Capturas del Sistema	7 - 12
5. Dificultades y Soluciones	13
6. Conclusión	14
7. Bibliografía y Webgrafía	15

1- Introducción:

Food Store es una aplicación web Full Stack desarrollada como Trabajo Práctico Integrador de la materia Programación III.

El objetivo principal del proyecto fue implementar un sistema de gestión de pedidos de comida que permita administrar categorías, productos y pedidos mediante un panel administrativo, así como brindar a los usuarios finales la posibilidad de registrarse, navegar por un catálogo de productos, gestionar un carrito de compras y realizar pedidos.

El desarrollo se realizó utilizando una arquitectura cliente-servidor compuesta por un frontend desarrollado con Vite, TypeScript, HTML y CSS, y un backend desarrollado con Spring Boot, Java y MySQL. La comunicación entre ambas capas se realiza mediante una API REST.

2- Marco Teórico:

Spring Boot

Spring Boot es un framework de desarrollo basado en Java que simplifica la creación de aplicaciones empresariales. Permite desarrollar servicios REST de forma rápida mediante configuraciones automáticas y una estructura organizada por capas.

Spring Data JPA

Spring Data JPA facilita la persistencia de datos mediante el uso de repositorios y entidades. Permite trabajar con bases de datos relacionales utilizando programación orientada a objetos y reduciendo la cantidad de código necesario para las operaciones CRUD.

MySQL

MySQL es un sistema gestor de bases de datos relacional utilizado para almacenar de forma persistente la información del sistema, incluyendo usuarios, categorías, productos y pedidos.

TypeScript

TypeScript es una extensión de JavaScript que incorpora tipado estático. Su utilización permite detectar errores durante el desarrollo y facilita el mantenimiento del código.

Vite

Vite es una herramienta moderna para el desarrollo frontend que proporciona un entorno de trabajo rápido y eficiente para aplicaciones web desarrolladas con TypeScript y JavaScript.

Swagger OpenAPI

Swagger se utilizó para documentar y probar los endpoints REST del backend, facilitando la validación y el testeo de las funcionalidades implementadas.

3- Decisiones Técnicas y Arquitectura:

Arquitectura General

La solución fue desarrollada siguiendo una arquitectura cliente-servidor.

Frontend

El frontend fue desarrollado utilizando:

- HTML5
- CSS3
- TypeScript
- Vite

Las funcionalidades fueron organizadas en módulos independientes para autenticación, catálogo de productos, carrito de compras, pedidos de usuario y panel administrativo.

Backend

El backend fue desarrollado utilizando:

- Java 17
- Spring Boot
- Spring Data JPA
- Hibernate
- MySQL

La estructura del proyecto se organizó en capas siguiendo buenas prácticas de desarrollo:

- Controllers
- Services
- Repositories
- DTOs
- Entities

Esta separación permitió mantener una adecuada organización del código y facilitar el mantenimiento de la aplicación.

Uso de DTOs

Se utilizaron DTOs (Data Transfer Objects) para separar las entidades de persistencia de los objetos expuestos por la API.

Esta decisión permitió controlar la información enviada y recibida por los endpoints, además de incorporar validaciones específicas para cada operación.

Manejo Global de Excepciones

Se implementó un manejador global de excepciones mediante la anotación `@ControllerAdvice`.

Esto permitió centralizar las respuestas de error del sistema, generando mensajes consistentes para situaciones como:

- Recursos inexistentes.
- Errores de validación.
- Conflictos por registros duplicados.
- Errores de negocio relacionados con stock o disponibilidad.

Eliminación Lógica

Para preservar la integridad histórica de los datos se implementó eliminación lógica mediante el atributo "eliminado".

Cuando un registro es eliminado, permanece almacenado en la base de datos pero deja de mostrarse en las consultas activas del sistema.

Persistencia del Carrito

El carrito de compras fue implementado utilizando localStorage.

De esta manera los productos seleccionados por el usuario permanecen almacenados incluso si el navegador se cierra o la página se recarga.

Seguridad

Las contraseñas de los usuarios se almacenan utilizando cifrado BCrypt antes de ser persistidas en la base de datos.

Además, el sistema distingue dos roles principales:

- ADMIN
- CLIENTE

Cada rol posee acceso únicamente a las funcionalidades correspondientes.

4- Capturas del sistema:

Autenticación

Figura 1. Pantalla de inicio de sesión.

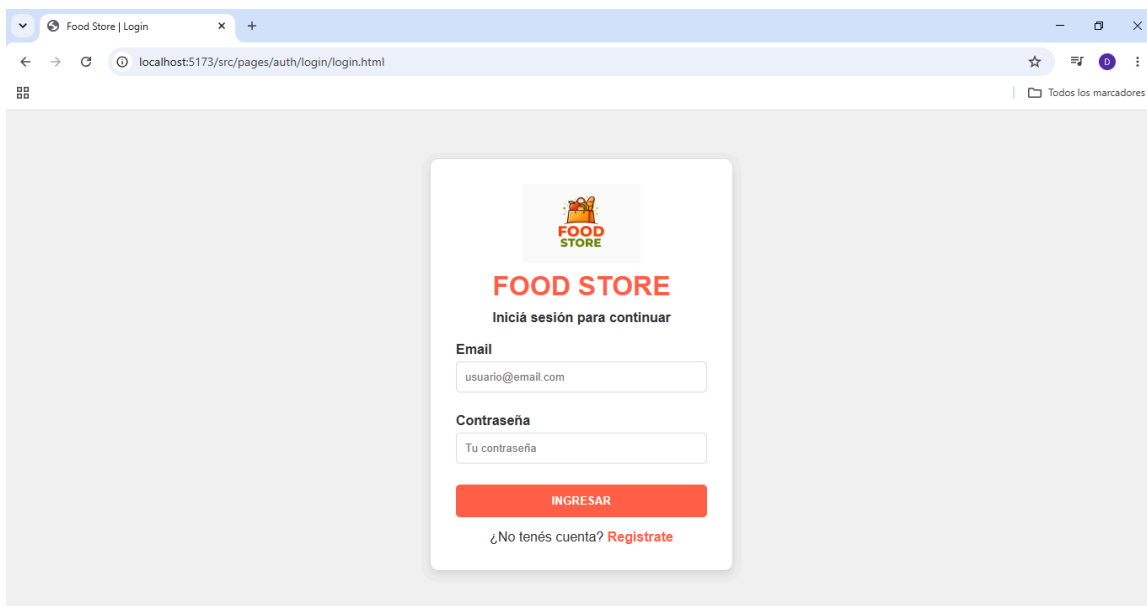
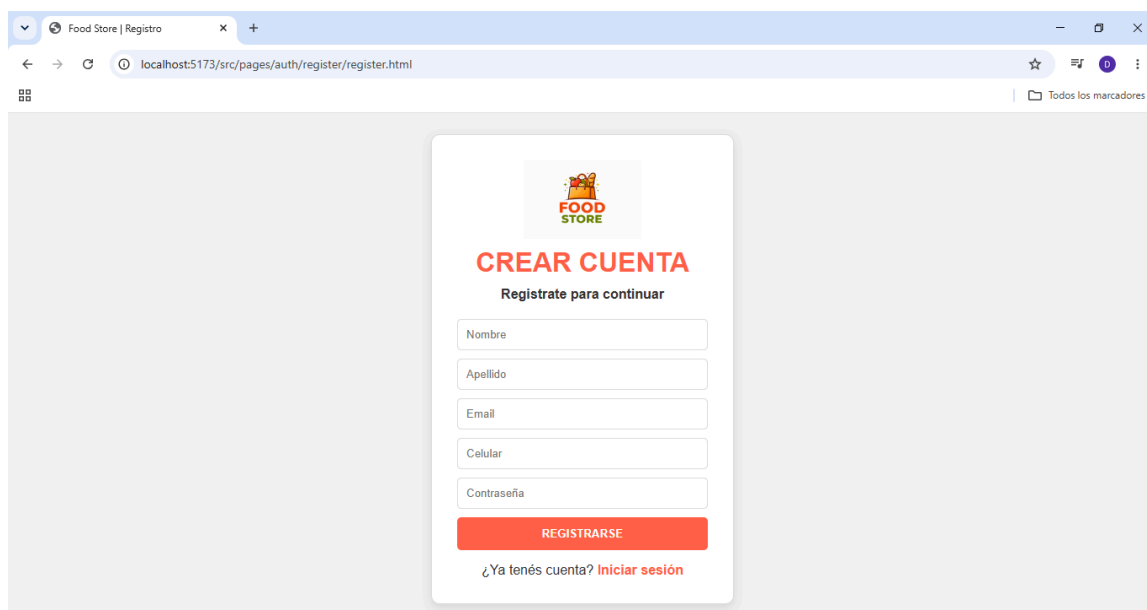


Figura 2. Pantalla de registro de usuario.



Cliente

Figura 3. Catálogo principal de productos.

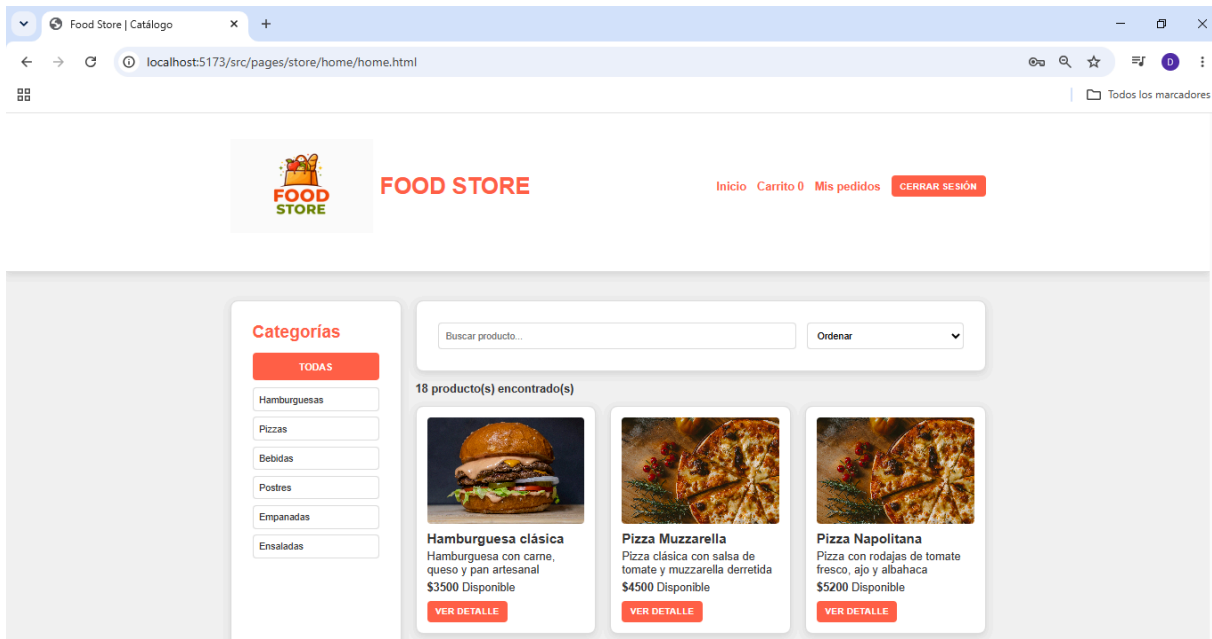


Figura 4. Detalle de producto.

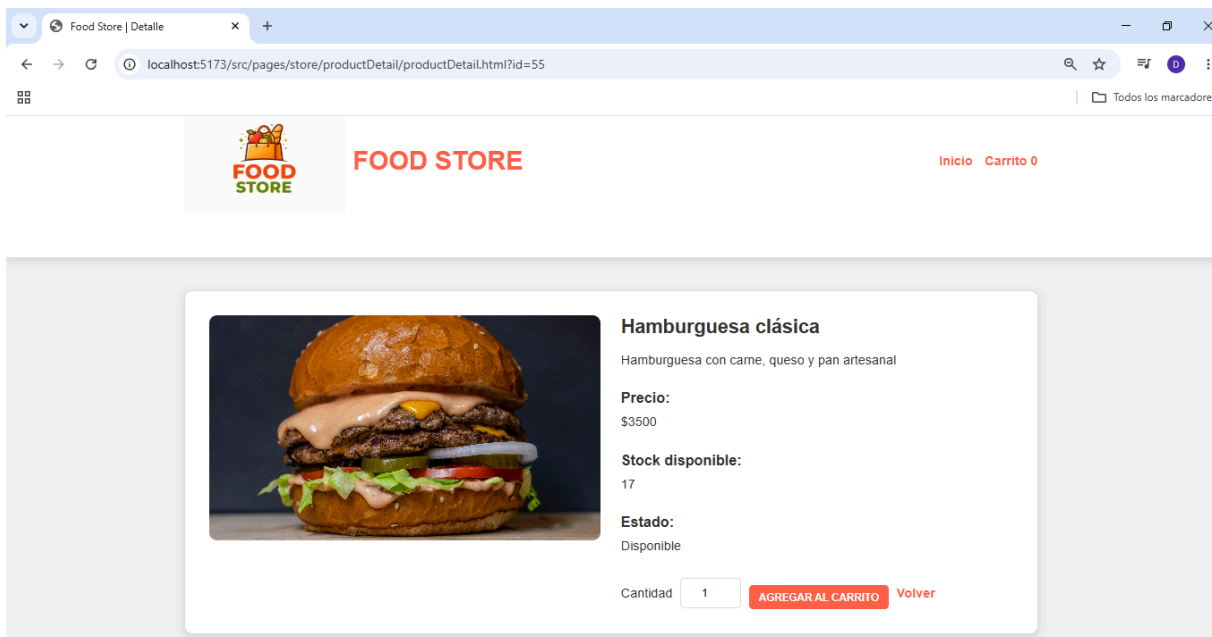


Figura 5. Carrito de compras.

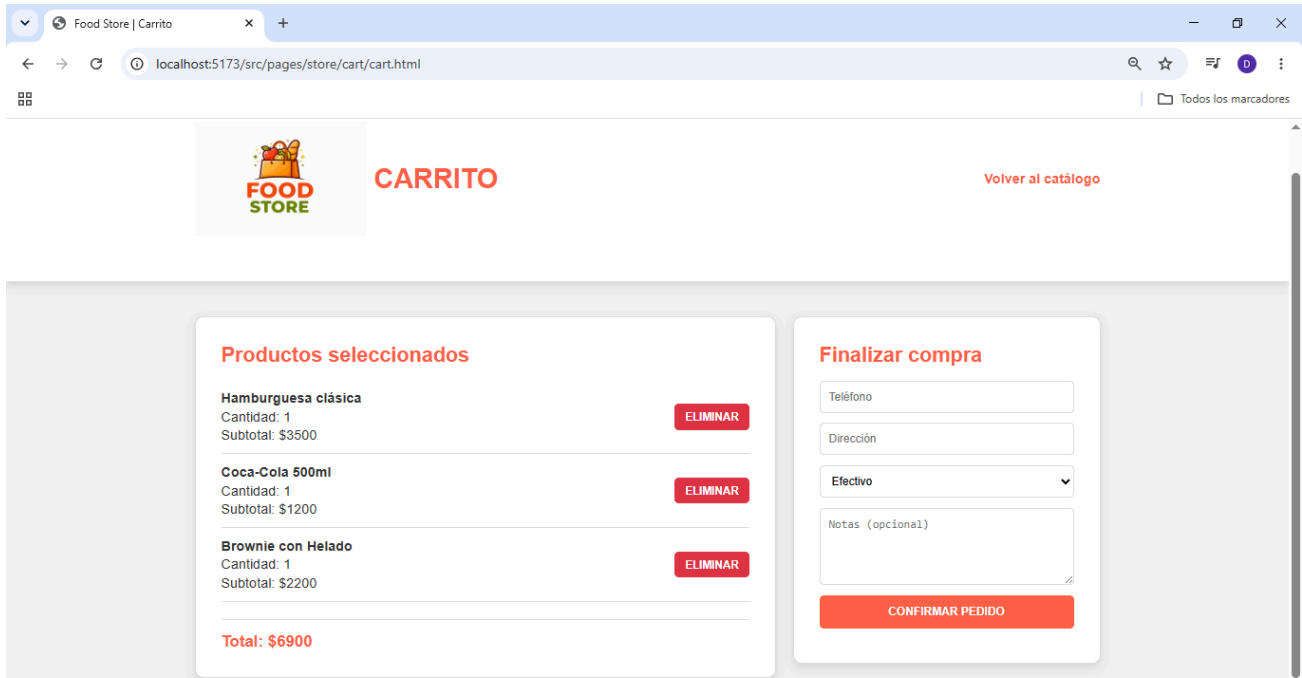


Figura 6. Historial de pedidos.



Administración

Figura 7. Dashboard administrativo.

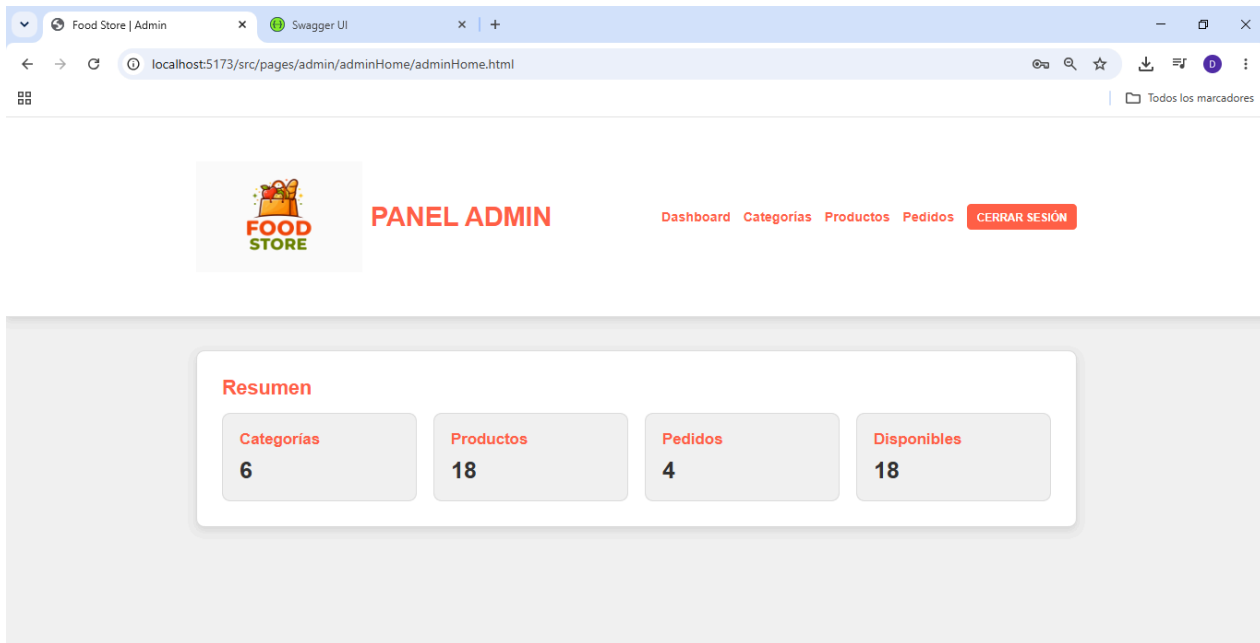


Figura 8. Gestión de categorías. (Se muestran las primeras 3)

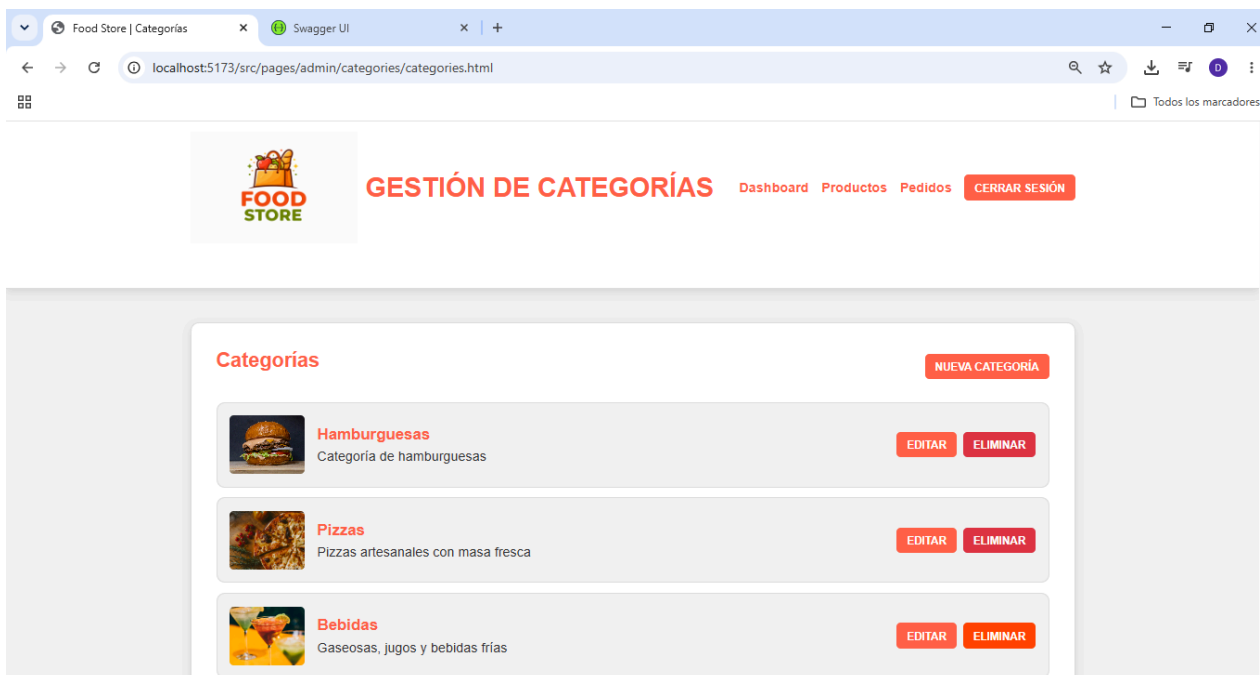


Figura 9. Gestión de productos. (Se muestran las primeras 3)

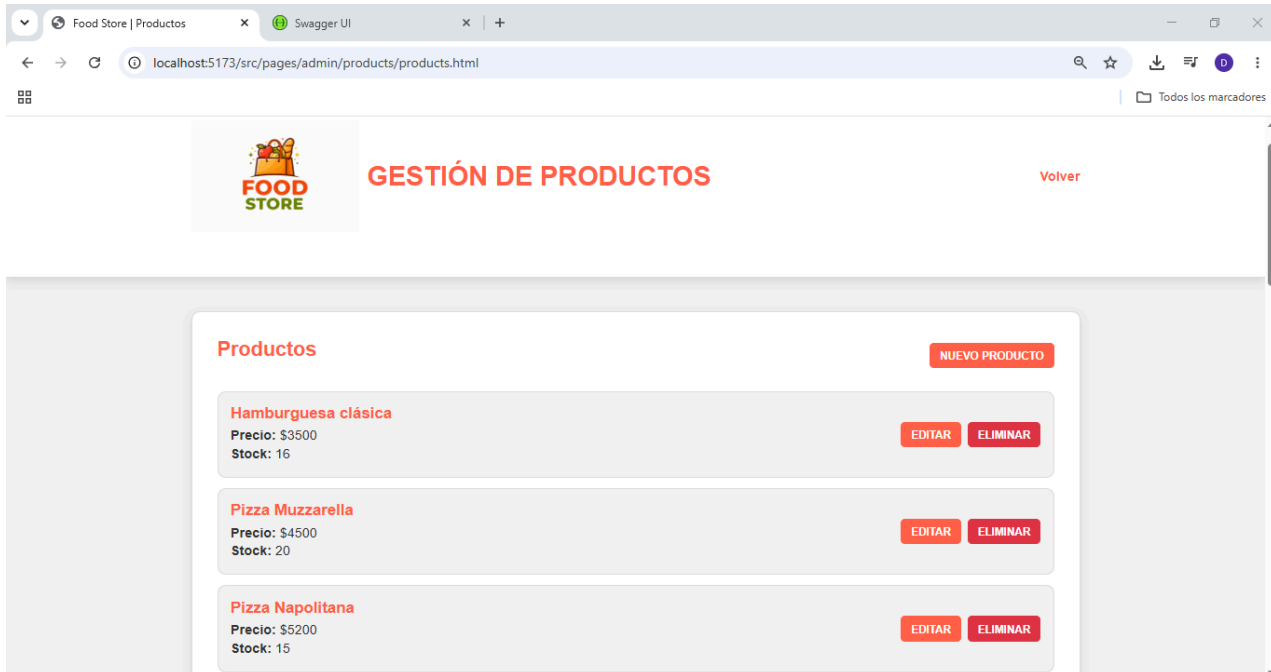
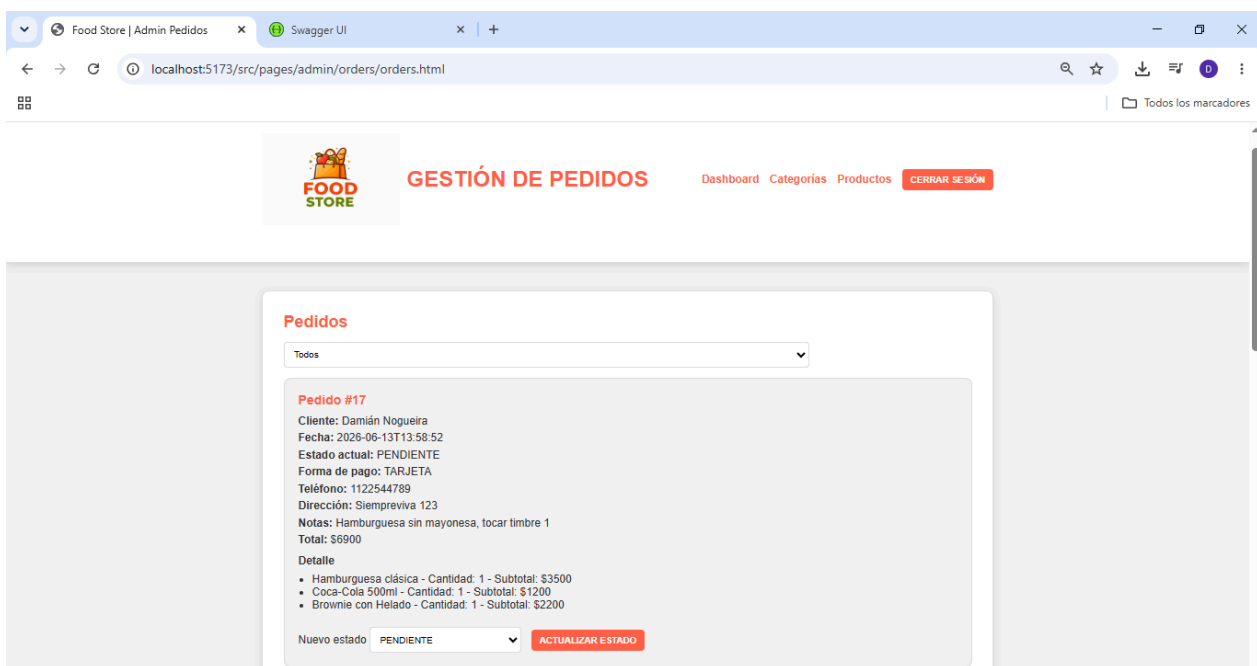


Figura 10. Gestión de pedidos. (Se muestra solo un pedido)



Backend

Figura 11. Documentación Swagger.

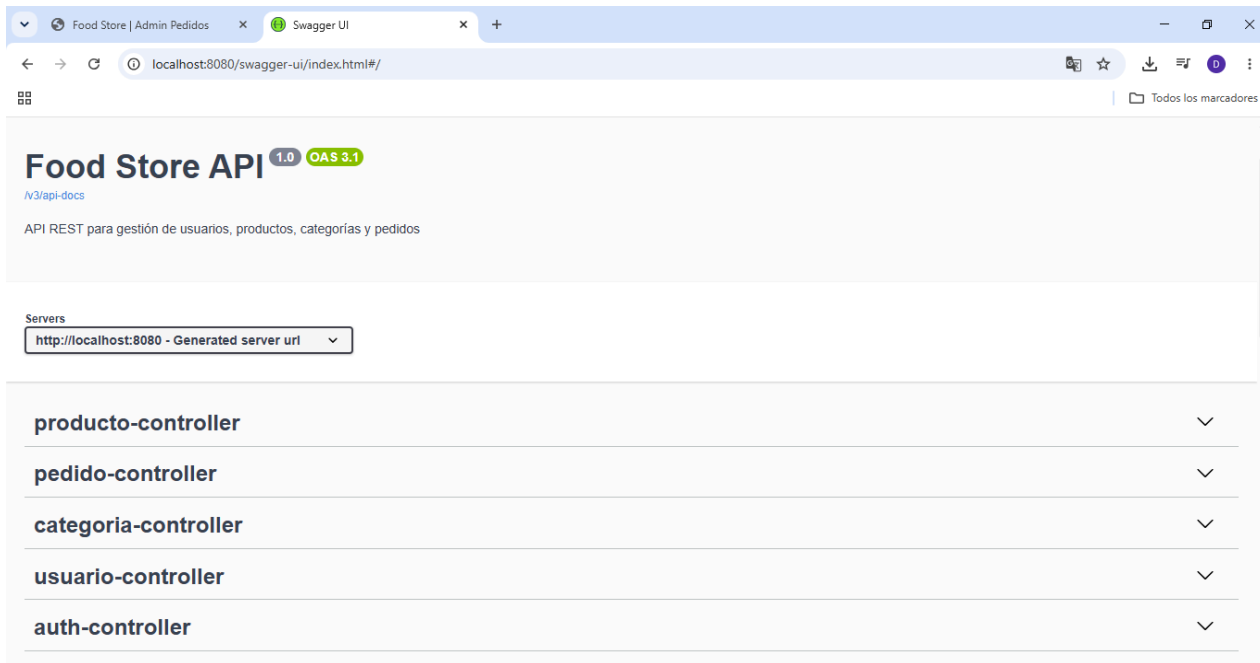
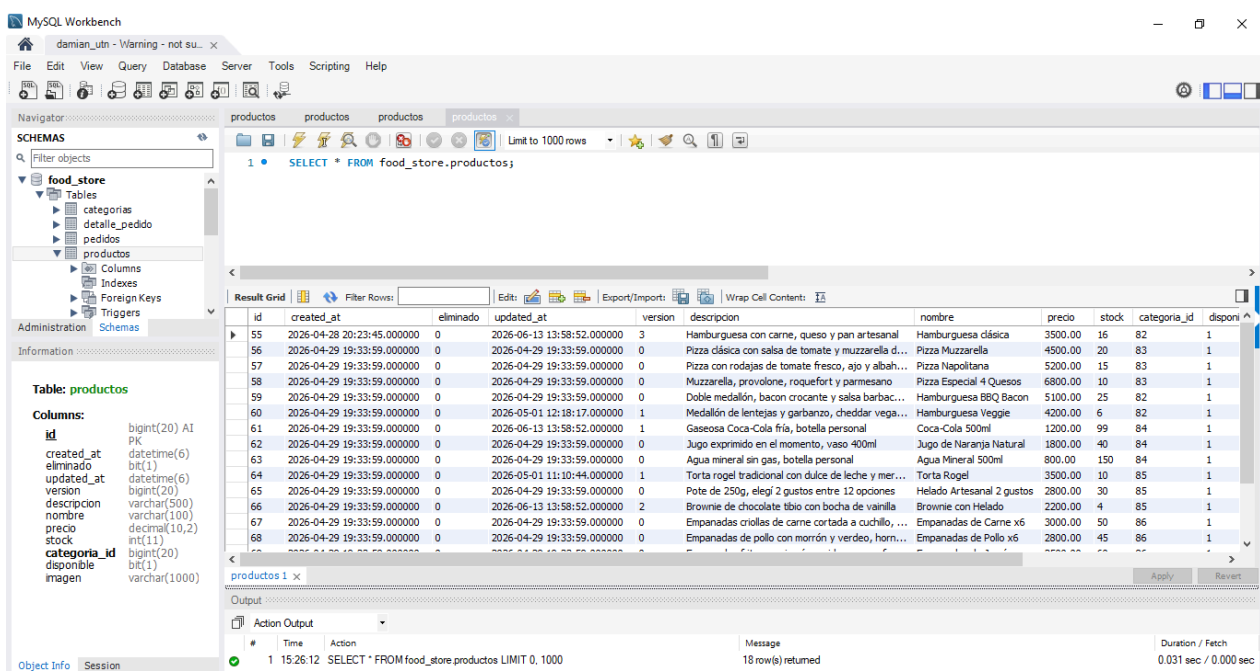


Figura 12. Base de datos MySQL.



5- Dificultades y Soluciones:

Integración entre Frontend y Backend

Durante el desarrollo fue necesario coordinar la comunicación entre ambas capas mediante peticiones HTTP.

Para resolverlo se implementó una capa centralizada de acceso a la API, permitiendo reutilizar lógica y simplificar el mantenimiento.

Gestión de Stock

Una de las principales dificultades fue garantizar que los pedidos respetaran el stock disponible de cada producto.

La solución consistió en incorporar validaciones dentro de la capa de servicios antes de confirmar cada compra.

Manejo de Errores

Inicialmente los errores eran tratados de forma individual en distintos controladores.

Posteriormente se implementó un manejador global de excepciones que permitió centralizar las respuestas y mejorar la consistencia de la API.

Persistencia del Carrito

Se requirió mantener el contenido del carrito entre distintas sesiones del navegador.

Para ello se utilizó localStorage, permitiendo conservar los productos seleccionados por el usuario incluso después de cerrar la aplicación.

6- Conclusión:

El desarrollo de Food Store permitió integrar y aplicar los conocimientos adquiridos a lo largo de la carrera en un proyecto Full Stack completo. Durante la realización del trabajo se implementaron funcionalidades de frontend y backend, incluyendo autenticación, gestión de categorías, productos y pedidos, persistencia de datos y comunicación mediante una API REST.

Asimismo, el proyecto permitió poner en práctica conceptos abordados en materias previas, especialmente Base de Datos, mediante el diseño del modelo de datos relacional, la utilización de MySQL y la gestión de entidades relacionadas. También se aplicaron conocimientos de programación orientada a objetos, estructuras de software por capas, validaciones y buenas prácticas de desarrollo.

La utilización de tecnologías como Spring Boot, Spring Data JPA, TypeScript y Vite permitió consolidar habilidades técnicas relacionadas con el desarrollo de aplicaciones web modernas, el manejo de bases de datos y la integración entre cliente y servidor.

Como resultado, se obtuvo una aplicación funcional que cumple con los requerimientos planteados en las historias de usuario y los objetivos establecidos para el Trabajo Práctico Integrador, representando una experiencia significativa para afianzar conocimientos y adquirir experiencia práctica en el desarrollo de software.

7- Bibliografía y Webgrafía:

Bibliografía

- Building REST Services with Spring. Spring Guides. Recuperado de: <https://spring.io/guides/tutorials/rest>
- REST With Spring Tutorial. Baeldung. Recuperado de: <https://www.baeldung.com/rest-with-spring-series>
- Apuntes de la asignatura Programación 3

Webgrafía

- Spring Boot Documentation. Recuperado de: <https://spring.io/projects/spring-boot>
- Spring Data JPA Documentation. Recuperado de: <https://spring.io/projects/spring-data-jpa>
- Hibernate ORM Documentation. Recuperado de: <https://hibernate.org>
- MySQL Documentation. Recuperado de: <https://dev.mysql.com/doc>
- TypeScript Documentation. Recuperado de: <https://www.typescriptlang.org>
- Vite Documentation. Recuperado de: <https://vitejs.dev>
- Swagger OpenAPI Documentation. Recuperado de: <https://swagger.io>
- MDN Web Docs. Recuperado de: <https://developer.mozilla.org>
- Videos de la asignatura Programación 3